

Name- Swapnil Pal

WEEK 5-6 Documentation

```
import numpy as np

sequence_length = 24
X_seq = []
y_seq = []

# Sort by Home ID and Date
df_sorted = df_ml.sort_values(by=["Home ID", "Date"])

features = X.columns
target = "Energy Consumption (kWh)"

# Convert all features to float
df_sorted[features] = df_sorted[features].astype(float)

for home in df_sorted["Home ID"].unique():

    home_data = df_sorted[df_sorted["Home ID"] == home]

    for i in range(len(home_data) - sequence_length):
        X_seq.append(home_data[features].iloc[i:i+sequence_length].values)
        y_seq.append(home_data[target].iloc[i+sequence_length])

X_seq = np.array(X_seq, dtype=np.float32)
y_seq = np.array(y_seq, dtype=np.float32)

print("New Sequence Shape:", X_seq.shape)
```

... New Sequence Shape: (80003, 24, 19)

This code prepares time-series sequences of length **24** for each home by sorting data by **Home ID** and **Date**, then applying a sliding window technique. It converts features to float, generates input sequences (**X_seq**) and corresponding target values (**y_seq**), and finally produces a dataset of shape **(80003, 24, 19)** for model training.

```

split = int(0.8 * len(X_seq))

X_train_seq = X_seq[:split]
X_test_seq = X_seq[split:]

y_train_seq = y_seq[:split]
y_test_seq = y_seq[split:]

print("Train shape:", X_train_seq.shape)
print("Test shape:", X_test_seq.shape)

Train shape: (64002, 24, 19)
Test shape: (16001, 24, 19)

```

This code splits the sequence dataset into **80% training** and **20% testing** using `split = int(0.8 * len(X_seq))`.

The training set contains **64,002 samples** and the test set contains **16,001 samples**, each with a shape of **(24, 19)**, indicating 24 time steps and 19 features per sample.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam

model = Sequential()
model.add(LSTM(128, return_sequences=True, input_shape=(X_train_seq.shape[1], X_train_seq.shape[2])))
model.add(Dropout(0.3))

model.add(LSTM(64))
model.add(Dropout(0.3))

model.add(Dense(32, activation='relu'))
model.add(Dense(1))

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
super().__init__(**kwargs)
Model: "Sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 24, 128)	75,776
dropout_1 (Dropout)	(None, 24, 128)	0
lstm_2 (LSTM)	(None, 64)	40,400
dropout_2 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,048
dense_3 (Dense)	(None, 1)	32

Total params: 117,216 (497.25 KB)
Trainable params: 117,216 (497.25 KB)
Non-trainable params: 0 (0.00 KB)

This code builds an **LSTM-based deep learning model** with two LSTM layers and dropout for predicting energy consumption from time-series data.

It uses **Adam optimizer**, **MSE loss**, and **MAE metric**, and outputs a single value, with a total of **127,297 trainable parameters**.

```

history = model.fit(
    X_train_seq,
    y_train_seq,
    epochs=40,
    batch_size=64,
    validation_split=0.1
)

... Epoch 1/40
901/901 ————— 64s 64ms/step - loss: 0.7726 - mae: 0.6931 - val_loss: 0.7502 - val_mae: 0.6665
Epoch 2/40
901/901 ————— 83s 65ms/step - loss: 0.7268 - mae: 0.6740 - val_loss: 0.7214 - val_mae: 0.6724
Epoch 3/40
901/901 ————— 61s 67ms/step - loss: 0.7177 - mae: 0.6738 - val_loss: 0.7308 - val_mae: 0.6659
Epoch 4/40
901/901 ————— 57s 64ms/step - loss: 0.7116 - mae: 0.6690 - val_loss: 0.7215 - val_mae: 0.6736
Epoch 5/40
901/901 ————— 57s 63ms/step - loss: 0.7315 - mae: 0.6799 - val_loss: 0.7304 - val_mae: 0.6659
Epoch 6/40
901/901 ————— 53s 59ms/step - loss: 0.7117 - mae: 0.6701 - val_loss: 0.7218 - val_mae: 0.6703
Epoch 7/40
901/901 ————— 74s 50ms/step - loss: 0.7107 - mae: 0.6709 - val_loss: 0.7219 - val_mae: 0.6702
Epoch 8/40
901/901 ————— 47s 52ms/step - loss: 0.7227 - mae: 0.6754 - val_loss: 0.7495 - val_mae: 0.6664
Epoch 9/40
901/901 ————— 46s 51ms/step - loss: 0.7191 - mae: 0.6740 - val_loss: 0.7402 - val_mae: 0.6657
Epoch 10/40
901/901 ————— 47s 52ms/step - loss: 0.7192 - mae: 0.6720 - val_loss: 0.7290 - val_mae: 0.6661
Epoch 11/40
901/901 ————— 48s 53ms/step - loss: 0.7225 - mae: 0.6764 - val_loss: 0.7242 - val_mae: 0.6678
Epoch 12/40
901/901 ————— 87s 59ms/step - loss: 0.7187 - mae: 0.6739 - val_loss: 0.7241 - val_mae: 0.6678
Epoch 13/40
901/901 ————— 56s 62ms/step - loss: 0.7209 - mae: 0.6750 - val_loss: 0.7236 - val_mae: 0.6681
Epoch 14/40
901/901 ————— 62s 69ms/step - loss: 0.7181 - mae: 0.6731 - val_loss: 0.7215 - val_mae: 0.6712
Epoch 15/40
901/901 ————— 62s 68ms/step - loss: 0.7151 - mae: 0.6730 - val_loss: 0.7261 - val_mae: 0.6669
Epoch 16/40
901/901 ————— 61s 67ms/step - loss: 0.7227 - mae: 0.6765 - val_loss: 0.7255 - val_mae: 0.6672

```

This code trains the LSTM model using the training sequences for **40 epochs** with a **batch size of 64** and a **10% validation split**.

The training and validation losses gradually decrease, showing that the model is learning patterns while maintaining stable performance without major overfitting.

```

y_pred_lstm = model.predict(X_test_seq)

lstm_mae = mean_absolute_error(y_test_seq, y_pred_lstm)
lstm_rmse = np.sqrt(mean_squared_error(y_test_seq, y_pred_lstm))
lstm_r2 = r2_score(y_test_seq, y_pred_lstm)

print("Improved LSTM Performance")
print("MAE:", round(lstm_mae,4))
print("RMSE:", round(lstm_rmse,4))
print("R2:", round(lstm_r2,4))

... 501/501 ————— 5s 10ms/step
Improved LSTM Performance
MAE: 0.6658
RMSE: 0.8419
R2: -0.0016

```

This code uses the trained LSTM model to predict energy consumption on the test dataset and evaluates its performance.

The results show **MAE = 0.6658**, **RMSE = 0.8419**, and **R² = -0.0016**, indicating moderate prediction error and weak explanatory power on unseen data.

```
comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Random Forest", "LSTM"],
    "MAE": [lr_mae, rf_mae, lstm_mae],
    "RMSE": [lr_rmse, rf_rmse, lstm_rmse],
    "R2 Score": [lr_r2, rf_r2, lstm_r2]
})

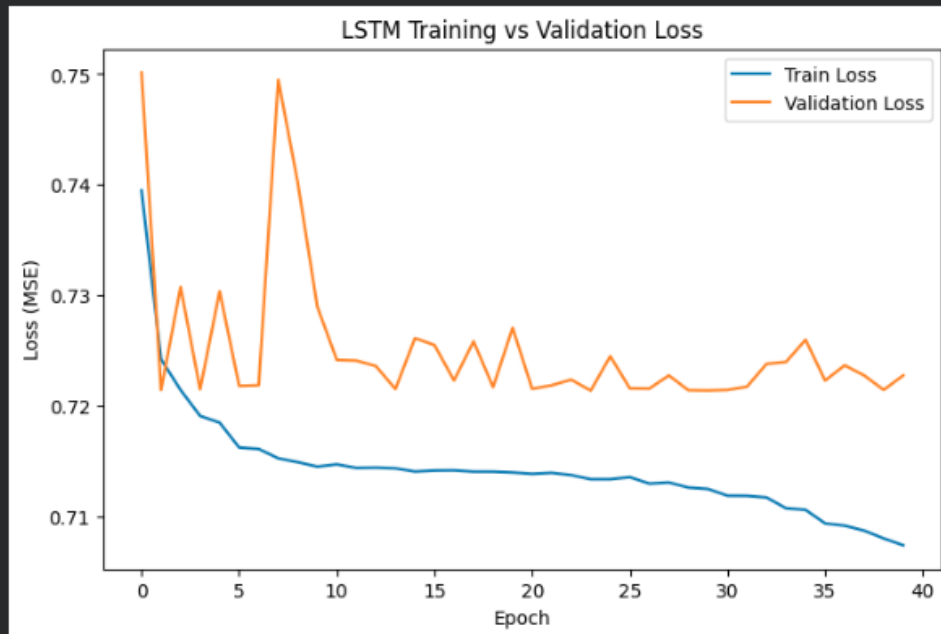
comparison
```

	Model	MAE	RMSE	R2 Score
0	Linear Regression	0.410420	0.490360	0.663869
1	Random Forest	0.417403	0.503100	0.646176
2	LSTM	0.665845	0.841876	-0.001570

This code creates a comparison table to evaluate **Linear Regression, Random Forest, and LSTM** models using MAE, RMSE, and R² scores.

The results show that **Linear Regression and Random Forest perform better** than LSTM, while the LSTM model has higher errors and a negative R², indicating poor predictive performance.

```
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("LSTM Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.show()
```



This plot shows the **training and validation loss (MSE)** of the LSTM model over 40 epochs. The training loss steadily decreases, while the validation loss remains almost constant with small fluctuations, indicating limited improvement on unseen data and possible underfitting.

```
import joblib

joblib.dump(lr_model, "best_energy_model.pkl")

['best_energy_model.pkl']

from google.colab import files
files.download("best_energy_model.pkl")
```

This code saves the trained Linear Regression model using **joblib** into a file named **best_energy_model.pkl**.

It then downloads the saved model from Google Colab, allowing it to be reused later for prediction without retraining.